

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ СИСТЕМ
УПРАВЛЕНИЯ И РАДИОЭЛЕКТРОНИКИ (ТУСУР)
Кафедра экономической математики, информатики и статистики (ЭМИС)

АЛГОРИТМЫ СОРТИРОВКИ И ПОИСКА

Отчёт по лабораторной работе по дисциплине «Программирование»

Студент гр.594-2

А А. А. Танцеров

“ 15 ” Марта 2025 г.

Руководитель канд. физ.- мат. наук,
доцент каф. ЭМИС ТУСУР

_____ Е. А. Шельмина

“ _____ ” _____ 2025 г.

Томск 2025

Лабораторная работа №6

Алгоритмы сортировки и поиска

Цель работы: приобретение навыков использования алгоритмов сортировки и поиска.

Исходный массив длиной N разбивается на две части: итог и остаток. Участок массива, называемый итогом, располагается с начала массива и должен быть упорядоченным, а участок массива, называемый остатком, располагается вплотную за итогом и содержит исходные числа не отсортированной части исходного массива.

Метод выборки включает в себя:

Шаг 1. Полагается $i=0$, т.е. считается, что итоговый участок - пуст.

Шаг 2. В остатке массива ищется минимальный элемент и он меняется местом с первым элементом остатка (i -ым элементом массива). После чего значение i увеличивается на единицу, тем самым расширяя итоговый участок массива (отсортированную часть исходного массива).

Шаг 3. Если $i < N$, то повторяется Шаг 2. В противном случае - конец алгоритма, т.к. итог становится равным всему массиву.

Конец алгоритма.

Метод пузырька: аналогично, как и в методе выбора, исходный массив длиной N разбивается на две части: отсортированную (итог) и не отсортированную (остаток). Первый элемент остатка является i -ым элементом массива.

Текстуальный алгоритм сортировки пузырьком:

Шаг 1. Пусть $k=N-1$, т.е. итоговый участок состоит из одного элемента.

Шаг 2. Берется первый элемент остатка и перемещается на место в итоговый участок массива так, чтобы итог остался упорядоченным. Первый элемент остатка назовем перемещаемым. Перемещение выполняется путем сравнения перемещаемого элемента с последующим элементом. Если

последующий элемент больше сравниваемого элемента, то процесс перемещения этого элемента закончен.

Шаг 3. После того, как первый элемент остатка переместился в итоговый участок, уменьшается на единицу значение переменной k , тем самым увеличивая отсортированную часть массива. Если $k > 0$, то управление передается на Шаг 2, в противном случае - работа алгоритма завершена.

Конец алгоритма.

Метод сортировки вставками

Этот метод похож на метод пузырька. Происходит такое же разбиение массива на отсортированную и не отсортированную части, но перемещение первого элемента остатка на принадлежащее ему место в итоге делается не сравнением двух соседних элементов, а с помощью метода двоичного поиска, который удобно оформить в виде отдельной процедуры. Текстуальный алгоритм методом включением:

Шаг 1. Пусть $i=1$, т.е. итоговый участок состоит из одного элемента.

Шаг 2. Берется первый элемент остатка и перемещается в отсортированную часть массива так, чтобы итоговый участок остался упорядоченным.

Шаг 3. После того, как первый элемент остатка переместился в итоговый участок, увеличивается на единицу значение переменной i , тем самым увеличивая отсортированную часть массива. Если $i < N$, то управление передается на Шаг 2, в противном случае - работа алгоритма завершена.

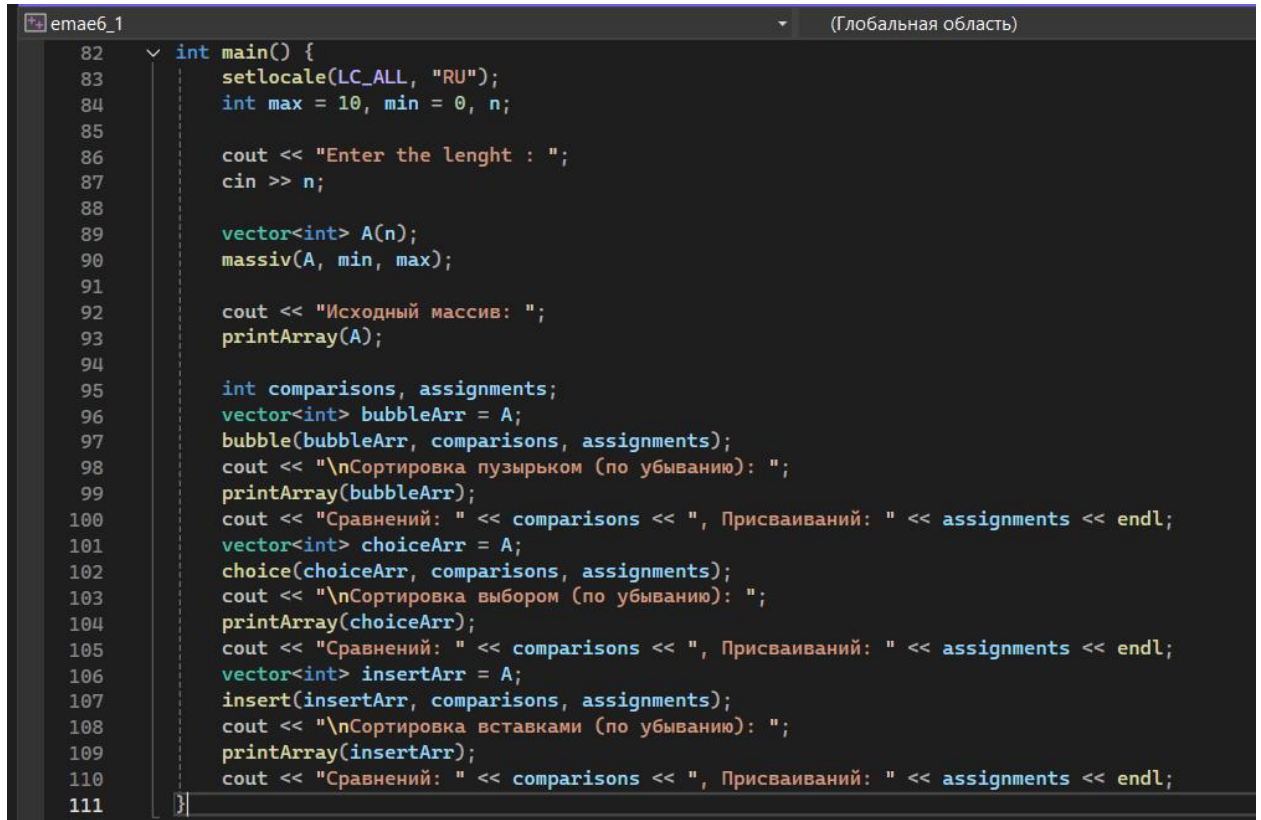
Конец алгоритма.

Задание 1. Написать программу для реализации трех алгоритмов сортировки (сортировка выбором, сортировка «пузырек», сортировка вставками). Каждый алгоритм должен быть оформлен в виде функции. Скриншот варианта представлен на рисунке 1.

11	Составить программу, которая для массива, заполненного случайными целыми числами, проводит сортировку по убыванию. Подсчитать количество присваиваний и количество сравнений при сортировке.
----	--

Рисунок 1 – Скриншот варианта к первому заданию

Используя библиотеку Vector, создаём динамический массив, который заполняем случайными числами, размер массива задан в программе. Создаём функцию PrintArray, которая выводит массив на экран. В main вызываем все наши функции, создаём массив и заполняем его. Результат представлен на рисунке 2.



```
82  int main() {
83      setlocale(LC_ALL, "RU");
84      int max = 10, min = 0, n;
85
86      cout << "Enter the lenght : ";
87      cin >> n;
88
89      vector<int> A(n);
90      massiv(A, min, max);
91
92      cout << "Исходный массив: ";
93      printArray(A);
94
95      int comparisons, assignments;
96      vector<int> bubbleArr = A;
97      bubble(bubbleArr, comparisons, assignments);
98      cout << "\nСортировка пузырьком (по убыванию): ";
99      printArray(bubbleArr);
100     cout << "Сравнений: " << comparisons << ", Присваиваний: " << assignments << endl;
101     vector<int> choiceArr = A;
102     choice(choiceArr, comparisons, assignments);
103     cout << "\nСортировка выбором (по убыванию): ";
104     printArray(choiceArr);
105     cout << "Сравнений: " << comparisons << ", Присваиваний: " << assignments << endl;
106     vector<int> insertArr = A;
107     insert(insertArr, comparisons, assignments);
108     cout << "\nСортировка вставками (по убыванию): ";
109     printArray(insertArr);
110     cout << "Сравнений: " << comparisons << ", Присваиваний: " << assignments << endl;
111 }
```

Рисунок 2 – Скриншот main программы

Создаём три функции, каждая из которых отвечает за свой метод сортировки, а именно: bubble, insert и choose. Кратко описав, сортировка пузырьком представляет собой сравнение двух соседних элементов, в зависимости от предназначения сортировки (по возрастанию или убыванию) меняем знак. Метод был назван так, потому что последний элемент «всплывает», то есть является отсортированным. Метод выбора представляет собой поиск минимального или максимального элемента массива, затем происходит замена текущего элемента и минимального, конечно, если это разные элементы. Вставка представляет собой сравнение двух элементов, если элемент больше или меньше, то вставляем данный элемент назад, иначе вперёд. Функции сортировок представлены на рисунке 3.

```

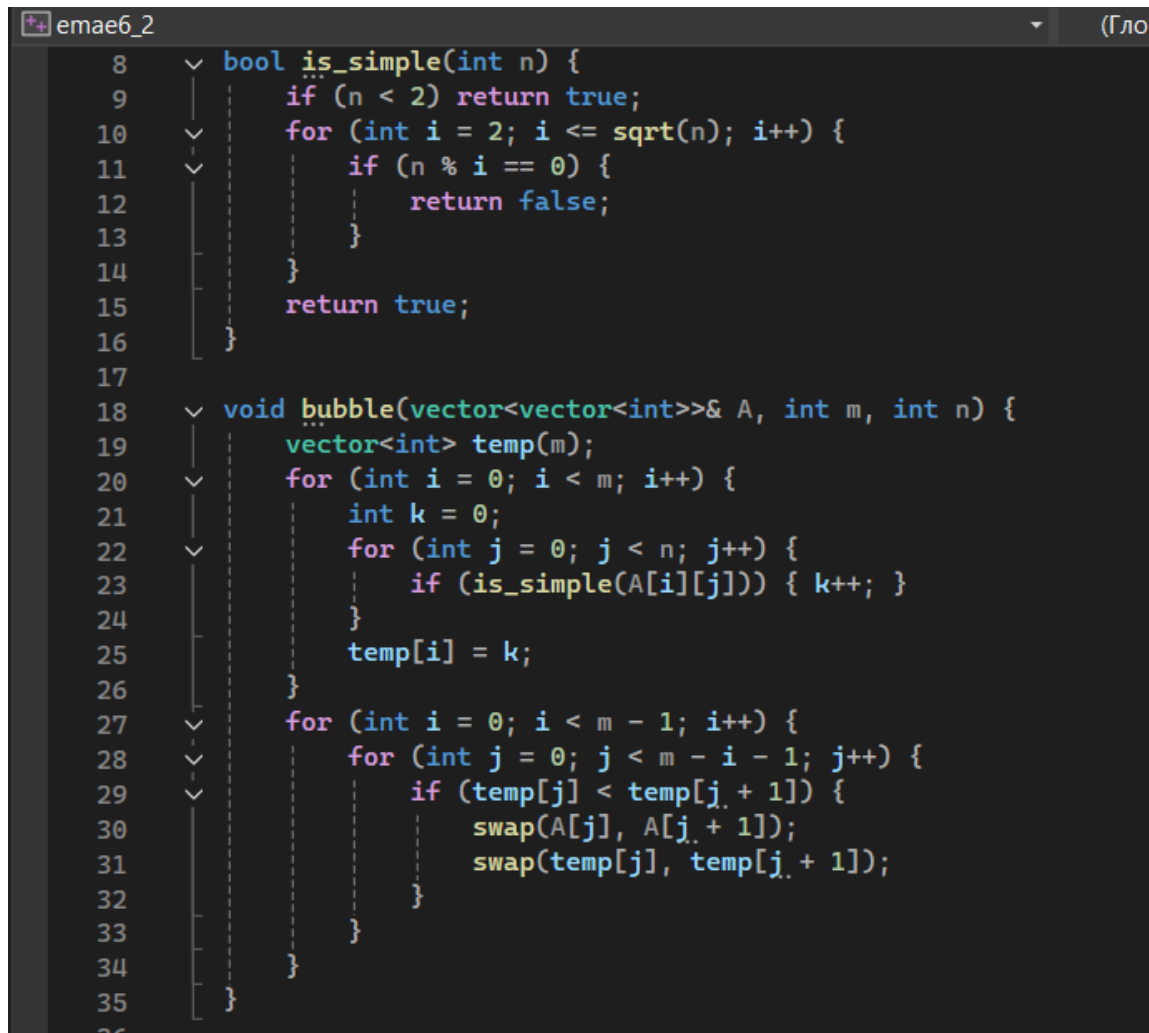
emaе6_1
7 void bubble(vector<int>& A, int& comparisons, int& assignments) {
8     int n = A.size();
9     comparisons = 0;
10    assignments = 0;
11
12    for (int i = 0; i < n - 1; i++) {
13        for (int j = 0; j < n - i - 1; j++) {
14            comparisons++;
15            if (A[j] < A[j + 1]) {
16                swap(A[j], A[j + 1]);
17                assignments += 3;
18            }
19        }
20    }
21 }
22 void choice(vector<int>& A, int& comparisons, int& assignments) {
23     int n = A.size();
24     comparisons = 0;
25     assignments = 0;
26
27     for (int i = 0; i < n - 1; i++) {
28         int index = i;
29         for (int j = i + 1; j < n; j++) {
30             comparisons++;
31             if (A[j] > A[index]) {
32                 index = j;
33                 assignments++;
34             }
35         }
36         if (index != i) {
37             swap(A[i], A[index]);
38             assignments += 3;
39         }
40     }
41 }
42 void insert(vector<int>& A, int& comparisons, int& assignments) {
43     int n = A.size();
44     comparisons = 0;
45     assignments = 0;
46
47     for (int i = 1; i < n; i++) {
48         int temp = A[i];
49         assignments++;
50         int j = i - 1;
51
52         while (j >= 0) {
53             comparisons++;
54             if (A[j] < temp) {
55                 A[j + 1] = A[j];
56                 assignments++;
57                 j--;
58             }
59             else {
60                 break;
61             }
62         }
63         A[j + 1] = temp;
64         assignments++;
65     }

```

Рисунок 3 – Скриншот запрограммированных методов сортировки

Задание 2. Дана целочисленная матрицы размера $n \times m$. Отсортировать элементы матрицы одним (любым) методом сортировки. Алгоритм сортировки оформить в виде функции. Переставить строки матрицы по убыванию количества простых чисел в строке.

Для выполнения условий задания, создаём функцию, которая проверяет, является ли число простым, то есть оно имеет делители единицу и само на себя, учитывая, что от нуля до двух уже являются простыми числами, начинаем поиск с двойки. Если число простое, то возвращаем True, иначе False. В качестве метода сортировки выбран метод сортировки пузырьком. Создаём функцию bubble, в которой создаём динамический массив, который заполнен количеством простых чисел каждой строки, затем методом пузырька производится проверка, если в следующей строчке простых чисел больше, то меняем местами. Выводим массив, в main опять всё вызываем. Функция bubble и is_simple представлена рисунке 4.



```
8  bool is_simple(int n) {
9      if (n < 2) return true;
10     for (int i = 2; i <= sqrt(n); i++) {
11         if (n % i == 0) {
12             return false;
13         }
14     }
15     return true;
16 }
17
18 void bubble(vector<vector<int>>& A, int m, int n) {
19     vector<int> temp(m);
20     for (int i = 0; i < m; i++) {
21         int k = 0;
22         for (int j = 0; j < n; j++) {
23             if (is_simple(A[i][j])) { k++; }
24         }
25         temp[i] = k;
26     }
27     for (int i = 0; i < m - 1; i++) {
28         for (int j = 0; j < m - i - 1; j++) {
29             if (temp[j] < temp[j + 1]) {
30                 swap(A[j], A[j + 1]);
31                 swap(temp[j], temp[j + 1]);
32             }
33         }
34     }
35 }
```

Рисунок 4 – Скриншот функций к заданию 2

Задание 3. выполнить задание с использованием алгоритма последовательного поиска и бинарного поиска. Оба алгоритма оформить в виде функций. Сформировать массив $a[n]$, упорядоченный по возрастанию.

Методом последовательного и бинарного поиска определить, содержит ли он заданное число. Если элемент не найден, то вставить его в массив на первое место. Небинарный метод заключается в поиске элемента последовательным перебором каждого элемента массива. Бинарный заключается в поочерёдном делении отрезка пополам, если элемент найден, то его возвращаем, иначе сравниваем, меньше он или больше, меняем диапазон поиска. Если элемент не найден вставляем данное число, которое ищем, в начало массива. Для бинарного поиска необходимо иметь отсортированный массив, выбор пал на пузырьки. Скриншот реализации функций представлен на рисунке 5.



```
емае6_3 (Глобальная)
6 void bubble(vector<int>& A, int n) {
7     for (int i = 0; i < n - 1; i++) {
8         for (int j = 0; j < n - i - 1; j++) {
9             if (A[j] > A[j + 1]) {
10                 swap(A[j], A[j + 1]);
11             }
12         }
13     }
14 }
15
16 vector<int> non_binary(vector<int> x, int target, int n) {
17     for (int i = 0; i < n; i++) {
18         if (x[i] == target) { return x; }
19     }
20     x.insert(x.begin(), target);
21     return x;
22 }
23
24 vector<int> binary(vector<int> x, int target, int n) {
25     int left = 0, right = x.size() - 1;
26     while (left <= right) {
27         int mid = left + (right - left) / 2;
28         if (x[mid] == target) {
29             return x;
30         }
31         if (x[mid] < target) {
32             left = mid + 1;
33         }
34         else {
35             right = mid - 1;
36         }
37     }
38     x.insert(x.begin(), target);
39     return x;
40 }
```

Рисунок 5 – Скриншот функций к заданию 3

Вывод: приобретены навыки использования алгоритмов сортировки и поиска.